

TS6 v4.1 — Lean 4 Certification of the Exact, Spectral, and Conditional Effective Dirichlet-Character Variance Layers

*Full formal closure of TS6-A/B/C/D and conditional closure of TS6-E/F/G
via a typed Large Sieve interface*

Serge Durand

Independent research project — TS Programme

Lean 4 toolchain v4.16.0, Mathlib v4.16.0

github.com/seisner1967-hash/ts6-lean-formalization

2026-05-03 (v4.1, revision 2.1)

Abstract

We report the closure of all five modules of the TS6 Lean 4 certification layer for the exact, spectral, and conditional effective Dirichlet-character variance identities of the TS programme. In version v4.1, the four exact and spectral modules (`FiniteCharacterVariance`, `DirichletVariance`, `CenteringShift`, `RankinSelbergDecomp`) are **GREEN**: closed in Lean without `sorry` and without added axioms. The fifth module (`AveragedBounds`) is **GREEN-conditional**: its three effective theorems `TS4_weighted`, `TS4_unweighted`, `TS3_first` are closed in Lean conditionally on a typed `LargeSieveInequality` interface and a `ChebyshevThetaBound` interface. The conditionality is reified as structure arguments rather than hidden behind `sorry` placeholders; `#print axioms` on the three effective theorems shows only Lean kernel axioms (`propext`, `Classical.choice`, `Quot.sound`), with no `sorryAx`.

The `LargeSieveInequality` structure is intentionally uninhabited within TS6: an instance must be supplied externally, either from a future Mathlib release exposing Iwaniec–Kowalski Theorem 7.13 or from an external proof bridge. TS6 v4.1 therefore does not prove the Large Sieve inequality, nor does it prove Goldbach’s conjecture or any conjecture of the TS programme; it certifies the implication chain from explicit analytic interfaces to the effective bounds.

Compared with TS6 v3 (2026-04-22), v4.1 closes the previously open Plancherel/Parseval kernel of `FiniteCharacterVariance`, the Rankin–Selberg decomposition of `RankinSelbergDecomp`, and the entire effective layer of `AveragedBounds`, while making the analytic boundary structurally explicit. The development is reproducible under Lean 4 v4.16.0 and Mathlib v4.16.0 (commit `a6276f4c`) via `lake build TS6`, with file SHA-256 hashes recorded in the freeze manifest `TS6_FROZEN_v4.1_2026-05-03.txt`. Source code, manifest, and audit reports are publicly archived at <https://github.com/seisner1967-hash/ts6-lean-formalization>, tag `v4.1`.

Contents

1	Introduction	3
2	Scope and Status of TS6 v4.1	4
2.1	What TS6 v4.1 is	4
2.2	What TS6 v4.1 is not	4
2.3	Top-level status table	4
2.4	Build summary	5

3	What changed: v3 $\rightarrow$ v4 $\rightarrow$ v4.1	5
3.1	AUX1: $\text{star}(\chi \ b) = (\chi \ b)^{-1}$ via root-of-unity argument	5
3.2	AUX2: dual Schur orthogonality on $\text{MulChar } G \ C$	5
3.3	<code>diag_offdiag_split</code> : diagonal/off-diagonal decomposition	5
3.4	<code>rankin_selberg_decomposition</code> : assembly of the spectral decomposition	6
3.5	<code>parseval_identity</code> : Plancherel for finite abelian groups	6
3.6	Closure of <code>AveragedBounds</code> as GREEN-conditional (v4 $\rightarrow$ v4.1)	6
4	Certified Exact Core: TS1 and TS2	7
4.1	Formal statement	7
4.2	Lean signatures (<code>DirichletVariance.lean</code>)	8
4.3	Why this is the central closure	8
5	Rankin–Selberg decomposition (TS6-D)	8
5.1	The decomposition	8
5.2	Lean signature and structure	9
5.3	Why v3 was quasi-GREEN	9
6	Plancherel/Parseval kernel (TS6-A)	9
6.1	The identity	9
6.2	Why v3 was YELLOW and v4 closes it	9
7	Conditional effective bounds (TS6-E/F/G)	10
7.1	The three effective theorems	10
7.2	The typed Large Sieve interface	10
7.3	The Chebyshev side	10
7.4	The cascade	10
7.5	Why <code>LargeSieveInequality</code> is uninhabited	11
8	Current limits of formalization	11
8.1	The reified analytic boundary	11
8.2	Why the conditionality is not hidden	11
8.3	Conditions for promotion to GREEN unconditional	11
9	Machine audit	12
10	Reproducibility	12
10.1	Environment	12
10.2	Public archive	12
10.3	Git trajectory of the closure	13
10.4	Trajectory of REAL sorry counts	13
11	Methodological gain	14
11.1	Detection of latent error in v3	14
11.2	Further methodological gain in v4.1: typing the analytic boundary	14
11.3	Other methodological observations	14
12	Conclusion	15
A	Lean file inventory	15

B	Key Lean signatures (verbatim from the source)	15
B.1	TS2 exact identity	15
B.2	Rankin–Selberg decomposition	15
B.3	Parseval/Plancherel for finite abelian groups	16
B.4	Typed Large Sieve interface	16
B.5	TS4 weighted effective bound (GREEN-conditional)	16
C	Freeze manifest excerpt	16
D	Reproduction commands	16

1 Introduction

The TS programme (TS1–TS5) develops an empirically-centered variance identity for smoothed prime indicators over arithmetic progressions, and a chain of effective bounds and spectral decompositions built on that identity. TS6 is not a new mathematical layer of that programme; rather, it is a *certification layer*: a Lean 4 project whose role is to formalize and machine-verify the central exact statements and their immediate spectral and effective consequences.

The present document records version v4.1 of that certification layer:

- **v2.2** (2026-04-21) reported a partial formalization framework with four remaining algebraic obligations.
- **v3** (2026-04-22) closed the two central exact identities `TS1_exact_identity` (prime modulus) and `TS2_exact_identity` (general $q \geq 2$).
- **v4** (2026-05-03 morning) closed the Rankin–Selberg decomposition (TS6-D) and the Parseval/Plancherel kernel (TS6-A), bringing the count of fully certified modules to four; the effective layer remained YELLOW by-design.
- **v4.1** (2026-05-03 freeze, this document) closes the effective layer (TS6-E/F/G) as GREEN-conditional via a typed `LargeSieveInequality` interface, eliminating all primary `sorry`s from the TS6 source.

The headline change in v4.1 is therefore not the addition of new mathematical content but a structural one: the analytic boundary that v3 and v4 documented as a textual “BY-DESIGN” comment is now reified as a typed Lean structure, statically enforced by the type system. The three effective theorems pass `#print axioms` with only Lean kernel axioms; the conditionality, formerly hidden behind `sorry`, is now visible in their signature.

Throughout this document, “closure” refers to Lean closure of the TS6 certification layer, not to an unconditional proof of the analytic inputs represented by the typed interfaces.

The rest of this paper is organized as follows. Section 2 delimits what TS6 v4.1 does and does not claim and presents the updated top-level status table. Section 3 documents the closures from v3 to v4 to v4.1, in six labeled sub-sections corresponding to the six closed obligations. Section 4 states the certified exact theorems and explains why they constitute the central closure. Sections 5 and 6 describe the closure of Rankin–Selberg and Parseval/Plancherel respectively. Section 7 describes the GREEN-conditional closure of the effective layer. Section 8 states the current limits of the formalization, including the conditions for promotion to GREEN unconditional. Section 9 records the machine audit, and Section 10 gives reproducibility data. Section 11 describes the methodological gains. Section 12 concludes.

2 Scope and Status of TS6 v4.1

2.1 What TS6 v4.1 is

TS6 v4.1 is a Lean 4 project that certifies five layers of the TS programme:

- the two exact identities $A_W^* = C_W$ for q prime (TS1) and $q \geq 2$ general (TS2);
- the centering-shift identity (Huygens decomposition);
- the Plancherel/Parseval identity for finite abelian groups;
- the Rankin–Selberg decomposition;
- three effective TS3/TS4 bounds, conditionally on typed analytic interfaces.

The certification of the first four is unconditional in the sense that no Lean axiom is introduced beyond those already present in Mathlib v4.16.0, and no `sorry` remains in the corresponding files. The fifth (effective bounds) is GREEN-conditional: closed in Lean without `sorry`, but parameterized by a typed `LargeSieveInequality` structure that TS6 itself does not inhabit.

2.2 What TS6 v4.1 is not

TS6 v4.1 is not, and does not claim to be:

- a complete formalization of TS1–TS5;
- a formalization of the TG programme;
- a proof of the Large Sieve inequality (this remains an external analytic input, now reified as a typed structure);
- a proof of the Chebyshev θ -bound (also external);
- a proof of Goldbach’s conjecture or the Riemann Hypothesis.

2.3 Top-level status table

Module / file	Layer	v4.1 status	REAL sorry
Exact/FiniteCharacterVariance.lean	TS6-A	GREEN	0
Exact/DirichletVariance.lean	TS6-B	GREEN	0
Exact/CenteringShift.lean	TS6-C	GREEN	0
Structure/RankinSelbergDecomp.lean	TS6-D	GREEN	0
Effective/AveragedBounds.lean	TS6-E/F/G	GREEN CONDITIONAL	0

Table 1: TS6 v4.1 module status: four modules GREEN unconditional, one GREEN conditional. All five close in Lean without `sorry`. The effective layer is closed conditionally on instances of the typed external interfaces `ChebyshevThetaBound` and `LargeSieveInequality`; the latter is intentionally uninhabited within TS6.

GREEN CONDITIONAL means closed in Lean without `sorry`, relative to explicit analytic interfaces. It does not mean the Large Sieve inequality is itself proved within TS6.

2.4 Build summary

Metric	Value
Lean version	leanprover/lean4:v4.16.0
Mathlib version	v4.16.0 (commit a6276f4c)
Build command	lake build TS6
Build status	SUCCESS (2.65 s)
Errors	0
Added axioms	0
Primary REAL sorry	0
sorryAx in cascade	NONE

Table 2: TS6 v4.1 build summary. Local development freeze at commit 7899dde; public archival release at commit 02a6ffa, tag v4.1.

3 What changed: v3 $\rightarrow$ v4 $\rightarrow$ v4.1

This section documents the six closures that bring TS6 from v3 to v4.1, in chronological order of the freeze. Each closure is presented with its mathematical role, its Lean strategy, and the corresponding Git commit.

3.1 AUX1: $\text{star } (\chi \ b) = (\chi \ b)^{-1}$ via root-of-unity argument

Role. In the dual Schur orthogonality lemma `schur_orthogonality_dual`, one needs the identity $\overline{\chi(b)} = \chi(b)^{-1}$ for $\chi : \text{MulChar } G \ \mathbb{C}$ and $b \in G$, where G is a finite abelian group.

Strategy. Mathlib’s `MulChar.star_apply`’ is blocked by a `[CommRing R]` constraint that does not hold for $R = G$ (a multiplicative `CommGroup`). The closure uses a manual root-of-unity argument: $\chi(b)^{|G|} = \chi(b^{|G|}) = \chi(1) = 1$, hence $\|\chi(b)\| = 1$, hence $\overline{\chi(b)} = \chi(b)^{-1}$.

Git. Commit 655908b. Approximately 16 lines of Lean.

3.2 AUX2: dual Schur orthogonality on $\text{MulChar } G \ \mathbb{C}$

Role. The dual orthogonality identity

$$\sum_{\chi : \text{MulChar } G \ \mathbb{C}} \chi(g) = \begin{cases} |G| & \text{if } g = 1, \\ 0 & \text{otherwise,} \end{cases}$$

required for the Rankin–Selberg decomposition.

Strategy. Mathlib has the analogous statement for `DirichletCharacter R q` at `Mathlib/NumberTheory/DirichletCharacter` but not directly for `MulChar G C`. The closure adapts that proof, mobilizing `MulChar.exists_apply_ne_one_of` (`Duality.lean:53`) and providing an instance of `HasEnoughRootsOfUnity C n` via `IsAlgClosed.hasEnoughRootsOfUnity`. The cardinality bookkeeping uses `MulChar.card_eq_card_units_of_hasEnoughRootsOfUnity`.

Git. Commit afaae39. Approximately 22 lines of Lean.

3.3 `diag_offdiag_split`: diagonal/off-diagonal decomposition

Role. The double sum appearing in the Rankin–Selberg decomposition,

$$\sum_{m,n \in S} [\text{Coprime}(m, q) \wedge \text{Coprime}(n, q) \wedge (m \equiv n \pmod{q})] a_m \overline{a_n},$$

splits as $(D_W : \mathbb{C}) + S_W$, where the diagonal piece D_W contains the $m = n$ contributions and the off-diagonal piece S_W contains $m \neq n$.

Strategy. Two private lemmas (`diag_part_eq_DW` and `offdiag_part_eq_SW`) handle each piece, then `Finset.sum_filter_add_sum_filter_not S (... = m)` recombines them. Coercion management between $\mathbb{R}$ and $\mathbb{C}$ uses `Complex.ofReal_sum` and the auxiliary identity `normSq_cast_eq_mul_c`.

Git. Commit 80ab482. Approximately 70 lines of Lean.

3.4 rankin_selberg_decomposition: assembly of the spectral decomposition

Role. The main theorem of TS6-D,

$$A_W^*(a, q, S) = D_W(a, q, S) - \frac{|T_0|^2}{\varphi(q)} + \operatorname{Re}(S_W(a, q, S)), \quad (1)$$

where $T_0 = \sum_r \theta(a, q, S, r)$.

Strategy. A private lemma `CW_eq_complex_decomp` performs steps 2–6 of the proof plan in $\mathbb{C}$ (isolation of the principal character via `filter_ne_one_eq_total_sub_one`, application of `sum_all_twistSum_sq` (`RS_L2`), application of `diag_offdiag_split` (`RS_L3`), and `field_simp; ring`). The final theorem then takes real parts via `congr_arg Complex.re`, `Complex.div_natCast_re`, the inverse direction of `Complex.ofReal_pow`, and `Complex.ofReal_re`.

Note. The v3 paper statement of the Rankin–Selberg decomposition omitted the explicit Re ; the v4 statement includes it because the right-hand side of the theorem lives in $\mathbb{R}$ while the off-diagonal sum S_W lives in $\mathbb{C}$. The Re extraction is performed in the final assembly step.

Git. Commit a5e2b84. Approximately 57 lines of Lean.

3.5 parseval_identity: Plancherel for finite abelian groups

Role. Parseval/Plancherel identity for $f : G \rightarrow \mathbb{C}$ with G a finite abelian group:

$$\operatorname{variance}(f) = \operatorname{characterSideMS}(f) = \frac{1}{|G|} \sum_{\chi \neq 1} |\hat{f}(\chi)|^2.$$

This identity is *not directly available in Mathlib v4.16.0* under the signature used here (with `MulChar G ℂ` as the dual side; Mathlib has the corresponding statement for additive characters into the unit circle, with a different formalism).

Strategy. Decomposed into four private lemmas:

1. `variance_eq_sum_sq_minus_card_meanSq` (König–Huygens identity) via `Complex.normSq_sub`;
2. `fourierCoeff_one_normSq` via `MulChar.one_apply` and `Group.isUnit`;
3. `sum_fourierCoeff_normSq_eq_card_mul_sum_normSq` via a local copy of `schur_orthogonality_dual` (the FCV ambient instances differ from those of RSD; consolidating into a shared upstream module is logged as future work), followed by a double `Finset.sum_comm` and a Schur orthogonality collapse;
4. `sum_filter_ne_one_eq_total_sub_real` (real-valued helper analogous to the complex helper from `rankin_selberg_decomposition`).

The main theorem assembles the four lemmas and concludes with `field_simp; ring`.

Git. Commits 8ac750f (preparatory lemmas) and 4f2000d (main theorem). Approximately 141 lines of Lean total.

3.6 Closure of AveragedBounds as GREEN-conditional (v4 → v4.1)

Role. The three effective theorems `TS4_weighted_effective_bound`, `TS4_unweighted_effective_bound`, and `TS3_first_effective_bound` of TS6-E/F/G, conditional on instances of `ChebyshevThetaBound` and `LargeSieveInequality`.

Strategy. In version v4, the file `AveragedBounds.lean` carried three placeholder `sorry`s, classified as YELLOW by-design, because the structure `LargeSieveInequality` held only the trivial field bound : `True`. Version v4.1 upgrades this layer in three steps:

1. **Typed Large Sieve interface.** The structure `LargeSieveInequality` is replaced by a single field bound carrying the ℓ^2 -form inequality

$$\text{weightedPrimeCappedSum}(a, Q, S) \leq (Q^2 + 2x) \cdot \text{coeffEnergy}(a, S),$$

for sequences $a : \mathbb{N} \rightarrow \mathbb{C}$ supported in a finite set S of indices bounded by $2x$. The new auxiliary definition $\text{coeffEnergy}(a, S) = \sum_{n \in S} \|a_n\|^2$ separates the character-side control (Large Sieve) from the coefficient-side control (Chebyshev). The structure is intentionally uninhabited within TS6.

2. **Coefficient energy bound (Chebyshev side).** A private lemma `coeffEnergy_weightCoeffs_le` bounds

$$\text{coeffEnergy}(\text{weightCoeffs}(W, x), S) \leq 4 \cdot \text{WsupNormSq}(W) \cdot x \cdot \log(2x)$$

using `ChebyshevThetaBound` together with the support estimate of W on $[1/2, 2]$ and the prime-counting trick $(\log p)^2 \leq \log(2x) \cdot \log p$.

3. **Cascade closure.** The three effective theorems are then closed by composition:

- `TS4_weighted` from `LargeSieveInequality.bound` and `coeffEnergy_weightCoeffs_le`;
- `TS4_unweighted` from `TS4_weighted` via the pointwise scaling $A_W^* \leq (q/2) \cdot A_W^*$ for $q \geq 2$ and $A_W^* \geq 0$;
- `TS3_first` from `TS4_unweighted` via the trivial monotonicity $X \leq 4C$ when $X \leq 2C$ and $C \geq 0$.

Audit. `#print axioms` on the three closed theorems reports only the Lean kernel axioms `propext`, `Classical.choice`, `Quot.sound`. No user-introduced axiom is added by TS6, and no `sorryAx` is transmitted through the cascade.

Git. Commit 516962b. Approximately 200 lines of Lean (including the typed interface, the energy bound, and three cascade proofs).

4 Certified Exact Core: TS1 and TS2

4.1 Formal statement

Let $q \geq 2$, $S \subset \mathbb{N}$ finite, $a : \mathbb{N} \rightarrow \mathbb{C}$ a weight, and $r \in (\mathbb{Z}/q\mathbb{Z})^\times$. Define the local prime sum

$$\theta(a, q, S, r) = \sum_{\substack{n \in S \\ n \equiv r \pmod{q}}} a_n,$$

the empirical mean

$$\bar{\theta}(a, q, S) = \frac{1}{\varphi(q)} \sum_{r \in (\mathbb{Z}/q\mathbb{Z})^\times} \theta(a, q, S, r),$$

and the empirically-centered variance

$$A_W^*(a, q, S) = \sum_{r \in (\mathbb{Z}/q\mathbb{Z})^\times} |\theta(a, q, S, r) - \bar{\theta}(a, q, S)|^2.$$

Let $C_W(a, q, S)$ denote the character-side sum, namely $1/\varphi(q)$ times the sum of $\|\text{twistSum}(a, q, S, \chi)\|^2$ over non-principal Dirichlet characters modulo q .

Theorem 4.1 (TS1, q prime — certified). *For every prime modulus q , every finite $S \subset \mathbb{N}$, and every weight $a : \mathbb{N} \rightarrow \mathbb{C}$,*

$$A_W^*(a, q, S) = C_W(a, q, S).$$

Theorem 4.2 (TS2, $q \geq 2$ — certified). *For every integer $q \geq 2$, every finite $S \subset \mathbb{N}$, and every weight $a : \mathbb{N} \rightarrow \mathbb{C}$,*

$$A_W^*(a, q, S) = C_W(a, q, S).$$

4.2 Lean signatures (DirichletVariance.lean)

```
theorem TS2_exact_identity
  (a : N -> C) (q : N) (hq : 2 <= q) (S : Finset N) :
  AWstar a q S = CW a q S := by
  -- Bloc A: AWstar = variance (theta a q S)
  -- Bloc B: variance = characterSideMS (via parseval)
  -- Bloc C: characterSideMS = CW (via mulchar_dirichlet_equiv)
  ...

theorem TS1_exact_identity
  (a : N -> C) (q : N) (hq : q.Prime) (S : Finset N) :
  AWstar a q S = CW a q S :=
  TS2_exact_identity a q hq.two_le S
```

The bijection between $\text{MulChar } (\mathbb{Z}\text{Mod } q)^{\times} \mathbb{C}$ and $\text{DirichletCharacter } \mathbb{C} \ q$ used in Bloc C is constructed as a noncomputable definition out of three standard Mathlib equivalences:

```
noncomputable def mulchar_dirichlet_equiv (q : N) [NeZero q] :
  MulChar (ZMod q)^x C ~ DirichletCharacter C q :=
  MulChar.equivToUnitHom.trans
  ((MulEquiv.monoidHomCongr (toUnits (G := (ZMod q)^x)).symm
    (MulEquiv.refl C^x)).toEquiv.trans
  MulChar.equivToUnitHom.symm)
```

4.3 Why this is the central closure

TS1 and TS2 are the exact identities on which every later layer of the TS programme rests. The Rankin–Selberg decomposition (TS5, formalized as TS6-D) applies the same character-side expansion with a different grouping of terms; the effective bounds (TS3, TS4, formalized as TS6-E/F/G) control A_W^* on average via C_W ; the centering-shift (TS6-C) expresses a Huygens-type decomposition that reduces to TS2 after one rewrite. Closing TS1 and TS2 in Lean therefore certifies the pivot of the programme.

5 Rankin–Selberg decomposition (TS6-D)

5.1 The decomposition

Theorem 5.1 (Rankin–Selberg — certified). *For every $q \geq 2$, every finite $S \subset \mathbb{N}$, and every weight $a : \mathbb{N} \rightarrow \mathbb{C}$,*

$$A_W^*(a, q, S) = D_W(a, q, S) - \frac{|T_0|^2}{\varphi(q)} + \text{Re}(S_W(a, q, S)),$$

where $T_0 = \sum_r \theta(a, q, S, r)$, and D_W, S_W are the standard Rankin–Selberg-type double sums.

5.2 Lean signature and structure

```

theorem rankin_selberg_decomposition
  (a : N -> C) (q : N) [NeZero q] (hq : 2 <= q) (S : Finset N) :
  AWstar a q S =
    DW a q S - (Complex.normSq (T_0 a q S)) / (Nat.totient q : R) + (SW a q S).re := by
  rw [TS2_exact_identity a q hq S]
  have hcomplex := CW_eq_complex_decomp a q S
  have h := congr_arg Complex.re hcomplex
  simp only [Complex.add_re, Complex.sub_re, Complex.div_natCast_re,
    Complex.ofReal_re] at h
  linarith [h]

```

The proof relies on three private lemmas: `CW_eq_complex_decomp` (the assembly in $\mathbb{C}$), `diag_part_eq_DW` and `offdiag_part_eq_SW` (the diagonal and off-diagonal pieces of the double sum), and the dual Schur orthogonality `schur_orthogonality_dual` with its two auxiliary lemmas (root-of-unity and Mathlib-Duality adaptation).

5.3 Why v3 was quasi-GREEN

In v3, three obligations remained open in `RankinSelbergDecomp.lean`:

- `L2_SCHUR_DUAL_CORE`, decomposed into `AUX1` and `AUX2`;
- `RS_L3 (diag_offdiag_split)`, the combinatorial split;
- `rankin_selberg_decomposition` itself, the final orchestration.

All three are now closed in v4.1, as documented in Sections 3.1, 3.2, 3.3, 3.4. The file is **GREEN**: 0 sorry.

6 Plancherel/Parseval kernel (TS6-A)

6.1 The identity

Theorem 6.1 (Parseval/Plancherel for finite abelian groups — certified). *For every finite abelian group G and every $f : G \rightarrow \mathbb{C}$,*

$$\text{variance}(f) = \text{characterSideMS}(f),$$

where $\text{variance}(f) = \sum_{g \in G} |f(g) - \text{mean}(f)|^2$ and

$$\text{characterSideMS}(f) = \frac{1}{|G|} \sum_{\chi \neq 1} |\hat{f}(\chi)|^2, \quad \hat{f}(\chi) = \sum_g f(g) \overline{\chi(g)}.$$

6.2 Why v3 was YELLOW and v4 closes it

The v3 paper noted that this identity “is logically independent of the closure of TS1 and TS2” — the dependency graph of `DirichletVariance.lean` does not pass through it. Nonetheless, closing it is desirable for two reasons: it removes the only remaining **sorry** on a primary file outside the by-design effective layer, and it provides a generic Plancherel kernel reusable in other parts of the development.

The closure proceeds via four private lemmas listed in Section 3.5. The hardest step is `sum_fourierCoeff_normSq_eq_card_mul_sum_normSq`, which performs the double sum manipulation

$$\sum_{\chi} |\hat{f}(\chi)|^2 = \sum_{\chi} \sum_{g, h} f(g) \overline{f(h)} \overline{\chi(g)} \chi(h) = \sum_{g, h} f(g) \overline{f(h)} \sum_{\chi} \overline{\chi(g)} \chi(h)$$

and collapses the inner sum via dual Schur orthogonality to $|G| \cdot [g = h]$, leaving $|G| \sum_g |f(g)|^2$. This is *not directly available in Mathlib v4.16.0* under this signature: Mathlib has Pontryagin duality for additive characters into the unit circle, but the multiplicative-character form on $\text{MulChar } G \mathbb{C}$ used here required a manual proof.

7 Conditional effective bounds (TS6-E/F/G)

7.1 The three effective theorems

The file `AveragedBounds.lean` carries three theorems controlling sums of A_W^* over prime moduli, each parameterized by typed external interfaces:

```
theorem TS4_weighted_effective_bound
  (chebyshev : ChebyshevThetaBound) (largeSieve : LargeSieveInequality)
  (Wi : WeightInput) (x : R) (Q : N) (S : Finset N)
  (hx : 2 <= x) (hQ : 2 <= Q)
  (hS : forall p in S, p.Prime /\ x/2 <= p /\ (p : R) <= 2*x) :
  weightedPrimeCappedSum (weightCoeffs Wi.W x) Q S <=
    4 * WsupNormSq Wi.W * ((Q : R)^2 + 2*x) * x * Real.log (2*x)
```

The unweighted variant divides the bound by 2, and the first-effective variant relaxes it to $4C$ from $2C$.

7.2 The typed Large Sieve interface

The structural change of v4.1 is that `LargeSieveInequality` is now a typed structure:

```
/-- Interface: large sieve inequality, packaged for prime moduli.
    The structure is intentionally uninhabited within TS6: an
    instance must be supplied by the caller, either from a future
    Mathlib release or from an external proof bridge. -/
structure LargeSieveInequality where
  bound :
    forall (a : N -> C) (x : R) (Q : N) (S : Finset N),
      2 <= x ->
        (forall n in S, ((n : R) <= 2*x)) ->
          weightedPrimeCappedSum a Q S <=
            ((Q : R)^2 + 2*x) * coeffEnergy a S
```

The companion definition $\text{coeffEnergy}(a, S) = \sum_{n \in S} \|a_n\|^2$ is the ℓ^2 -norm of the coefficient sequence on S ; it separates the character-side control (carried by the typed Large Sieve) from the coefficient-side control (carried by Chebyshev's θ -bound).

7.3 The Chebyshev side

The Chebyshev structure remains as in v3:

```
structure ChebyshevThetaBound where
  bound : forall (y : R), 2 <= y ->
    (sum p in (Finset.range (floor y + 1)).filter Nat.Prime, Real.log p) <= 2*y
```

The proof of `coeffEnergy_weightCoeffs_1e` — bounding `coeffEnergy` on the specific sequence `weightCoeffs(W, x)` — mobilizes `chebyshev.bound` together with the support estimate of W on $[1/2, 2]$ and the elementary inequality $(\log p)^2 \leq \log(2x) \cdot \log p$ for $p \in [x/2, 2x]$.

7.4 The cascade

```
-- TS4_weighted: from largeSieve.bound + coeffEnergy_weightCoeffs_le.
-- TS4_unweighted: from TS4_weighted via AWstar_nonneg + (q/2) >= 1.
-- TS3_first: from TS4_unweighted via 2C <= 4C when C >= 0.
```

Each step of the cascade is a few lines of Lean: `nlinarith` or `mul_le_mul_of_nonneg_left` usually suffices once the positivity of the relevant factor is established.

7.5 Why LargeSieveInequality is uninhabited

Constructing a mathematically natural instance of `LargeSieveInequality` would amount to formalizing an appropriate prime-moduli ℓ^2 -form of the large sieve inequality, after the collapse $q/\varphi(q) \cdot \sum_{\chi}^* = q$ for prime q which uses TS2 and the principal-character cancellation. This is a substantial result of analytic number theory closely related to standard formulations (cf. Iwaniec–Kowalski, *Analytic Number Theory*, Theorem 7.13), which is *not currently exposed in Mathlib v4.16.0* under the form required here. The TS6 interface is specialized to the prime-moduli case and to a specific positivity/support structure on the coefficient sequence; an inhabitant could come from a future Mathlib release, from an external proof bridge, or from an ad hoc derivation tailored to TS6. TS6 v4.1 does not attempt any of these; it formalizes the implication chain “Large Sieve interface $\Rightarrow$ effective bounds” and leaves the construction of an instance to a future development.

8 Current limits of formalization

8.1 The reified analytic boundary

The structures `LargeSieveInequality` and `ChebyshevThetaBound` are the formal interfaces to the two external analytic inputs. TS6 v4.1 does not provide an inhabitant of either: both must be supplied by a caller. The `ChebyshevThetaBound` interface is in principle inhabitable from Mathlib’s existing prime-counting layer (Mertens-style estimates exist in `Mathlib.NumberTheory.Mertens`), but this construction is not part of TS6. The `LargeSieveInequality` interface is genuinely external: no version of the prime-moduli ℓ^2 -form is currently exposed in Mathlib v4.16.0.

8.2 Why the conditionality is not hidden

A naive alternative would have been to close the second and third effective theorems (`TS4_unweighted`, `TS3_first`) by cascade from `TS4_weighted` while leaving `TS4_weighted` itself with a `sorry`. This would have reduced the visible `sorry` count from three to one, but at the cost of transmitting `sorryAx` through the cascade: `#print axioms` on `TS3_first` would have shown `sorryAx` as a transitive dependency.

The chosen design avoids this cosmetic improvement. By reifying the analytic input as a typed structure, we guarantee that:

- the three effective theorems are closed by Lean’s kernel without `sorryAx`;
- their dependency on the analytic input is statically visible in their signature as a structure argument;
- supplying an instance of the structure — once Mathlib makes one available — closes all three theorems unconditionally with no further modification of TS6.

8.3 Conditions for promotion to GREEN unconditional

Promotion of the effective layer from GREEN-conditional to GREEN unconditional requires either:

1. A future Mathlib release exposing the prime-moduli ℓ^2 -form of the large-sieve inequality, sufficient to construct an instance of `LargeSieveInequality`; together with an instance of `ChebyshevThetaBound` from Mathlib’s prime-counting layer; or
2. An ad hoc proof of either or both inequalities written specifically for TS6, supplied as a separate Lean module that constructs the instances.

Either path is outside the scope of the present version.

9 Machine audit

The TS6 source is audited by an external Python tool, `horizon-proof-agent v0.1.4`, which scans Lean files for `sorry/admit/axiom/unsafe` patterns after stripping comments and strings. The tool also runs `lake build TS6` and captures stdout/stderr.

The audit tool is not a proof assistant. Lean is the verifier. The tool reports the state of the Lean project and guards against stale headers, stale claims, and untracked proof obligations.

The final audit report `report_20260503T124108Z.md` confirms:

- Build status: SUCCESS (2.65 s);
- Primary REAL `sorry`: 0;
- Primary REAL axiom declarations: 0;
- No `sorryAx` in the cascade of effective theorems.

The output of `#print axioms` on the three GREEN-conditional theorems is reproduced verbatim:

```
'TS6.Effective.TS4_weighted_effective_bound' depends on axioms:
[propext, Classical.choice, Quot.sound]
'TS6.Effective.TS4_unweighted_effective_bound' depends on axioms:
[propext, Classical.choice, Quot.sound]
'TS6.Effective.TS3_first_effective_bound' depends on axioms:
[propext, Classical.choice, Quot.sound]
```

These are the three standard Lean kernel axioms (propositional extensionality, classical choice, quotient soundness), used internally throughout Mathlib. No `sorryAx` appears, confirming that the effective theorems are truly Lean-closed conditionally on their structure arguments.

10 Reproducibility

10.1 Environment

Component	Value
Lean toolchain	leanprover/lean4:v4.16.0
Mathlib revision	a6276f4c6097675b1cf5ebd49b1146b735f38c02 (v4.16.0)
Build tool	lake
Build command	lake build TS6

10.2 Public archive

Source code, freeze manifests (`TS6_FROZEN_v4_2026-05-03.txt` for v4 and `TS6_FROZEN_v4.1_2026-05-03.txt` for v4.1), and audit reports are publicly available at:

<https://github.com/seisner1967-hash/ts6-lean-formalization>

The state described in this document corresponds to tag **v4.1** on that repository. We distinguish two related commit identifiers:

- **7899dde** — final commit of the local development repository (**goldbach_lean_v2**), where the closure trajectory of Section 10.3 was carried out.
- **02a6ffa** — initial commit of the public archival repository (**ts6-lean-formalization**), a clean export of the v4.1 state, tagged **v4.1**.

The two commits correspond to the same Lean source state for the five TS6 modules; the public repository excludes scratch files, internal logs, and the obsolete v4 LaTeX draft, retaining only what is needed for independent reproduction (Lean sources, manifests, audit report, README, and licenses).

10.3 Git trajectory of the closure

Commit	Description
828e60e	Snapshot before closures
655908b	AUX1 closed (root of unity)
afaae39	AUX2 closed (Mathlib MulChar.Duality adaptation)
80ab482	diag_offdiag_split closed
a5e2b84	rankin_selberg_decomposition closed
8ac750f	Parseval Steps 1+3 prepared
4f2000d	parseval_identity complete
cd22e48	Session bilan + lakefile cleanup
36685bb	Freeze TS6 v4 (YELLOW BY-DESIGN)
b6d3257	Clarify v4 freeze manifest
2abc093	Add TS6 v4 LaTeX document (later superseded)
8b14291	Revise TS6 v4 LaTeX (later superseded)
516962b	Close AveragedBounds GREEN-conditional
806acad	Freeze TS6 v4.1
7899dde	Update session bilan: Phase 3 reached
02a6ffa	Initial public release of TS6 v4.1 (GitHub)

Table 3: Trajectory of the $v3 \rightarrow v4 \rightarrow v4.1$ closure session (2026-05-03).

10.4 Trajectory of REAL sorry counts

The closure session reduced the primary REAL **sorry** count along the following trajectory:

$$8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 4 \rightarrow 3 \rightarrow 0,$$

where each arrow corresponds to a single closure (the duplicated 4 reflects a partial Parseval commit before the full closure). The final value 0 corresponds to the GREEN-conditional state: all primary **sorry**s have been replaced by genuine proofs, with the remaining conditionality reified as typed structure arguments.

11 Methodological gain

11.1 Detection of latent error in v3

The v3 paper documented that the formalization detected and forced the correction of a latent error in an intermediate pointwise step in the TS2 proof: an originally stated norm equality

$$|\widehat{\theta}(\chi)| \stackrel{?}{=} |\text{twistSum}(a, q, S, \chi)|$$

turned out to be false at the complex level; the correct intermediate statement involves character inversion,

$$\widehat{\theta}(\chi) = \text{twistSum}(a, q, S, \chi^{-1}),$$

with the norm identity following as a corollary. We retain this observation here, as it remains the most striking instance of the formalization functioning as a logical audit rather than a passive transcription.

11.2 Further methodological gain in v4.1: typing the analytic boundary

Beyond the closures of v4 (Rankin–Selberg, Parseval/Plancherel), version v4.1 contributes a small but precise methodological observation. The original v4 design carried the conditionality of the effective layer as three `sorry` placeholders, classified textually as “BY-DESIGN” in source comments. Although correct in spirit, this design has a weakness: the conditionality is documented but not statically enforced — a reader unfamiliar with the convention could miss the comment, or a refactoring could silently broaden a placeholder beyond its intended scope.

Version v4.1 replaces this textual discipline by a structural one: the analytic input is reified as a Lean structure type, and its dependency appears in the signature of every theorem that uses it. This makes the conditionality *checked by the type system* rather than enforced by documentation. The gain is small but cumulative: every future refinement of TS6, and every external proof bridge that supplies an instance, will automatically respect the boundary.

The technique is not novel; it is the standard Lean idiom for “the next theorem assumes a typed external input”. We record it here because, in the context of an analytic-number-theoretic Lean development, the temptation to use `sorry` as a soft placeholder is strong, and the structural alternative is sometimes overlooked.

11.3 Other methodological observations

Three principles recurred throughout the v4 and v4.1 work:

Paper before Lean. The proof blocs that closed quickly are those that had been verified end-to-end on paper before any tactic was typed. The single instance in which this rule was broken (during the AUX2 attempt) led to a failed scratch that cost roughly thirty minutes; the subsequent paper-first version closed in fifteen.

Decomposition of obligations. Every closure of substantial size in this session was decomposed into named sub-obligations: AUX1+AUX2 for the dual Schur orthogonality, four private lemmas for Parseval, three sub-lemmas for the diagonal/off-diagonal split, the typed Large Sieve interface and the coefficient energy bound for the effective layer. Monolithic proof attempts failed opaquely; decomposed ones surfaced specific obstacles that could be addressed locally.

Audit before commit. Every closure was followed by a full audit run (`lake build TS6 + horizon-proof-agent`) before the corresponding Git commit. The audit reports document the trajectory $8 \rightarrow 0$ with each intermediate state machine-verified.

12 Conclusion

TS6 v4.1 closes all five modules of the Lean 4 certification layer for the exact, spectral, and conditional effective Dirichlet-character variance identities of the TS programme: four are **GREEN** unconditional and one is **GREEN CONDITIONAL**. The four exact and spectral modules (TS6-A through TS6-D) are GREEN: closed without **sorry** and without added axioms. The fifth module (TS6-E/F/G, effective bounds) is GREEN-conditional: closed without **sorry** but parameterized by typed external interfaces **ChebyshevThetaBound** and **LargeSieveInequality**. The latter is intentionally uninhabited within TS6 and represents the formal interface to the analytic content of the large sieve inequality.

The status of the formalization with respect to the broader TS programme can be stated precisely:

- **TS6 v4.1 is sorry-free in its certified Lean source.**
- **TS6 v4.1 closes the cascade “analytic interfaces $\Rightarrow$ effective bounds” relative to two typed inputs.**
- **TS6 v4.1 does not prove the Large Sieve inequality, Chebyshev’s θ -bound, Goldbach’s conjecture, or any conjecture of the TS programme.**

The development is reproducible, auditable, and publicly archived. The boundary between what is formally proved within TS6 and what is left to external analytic input is, in v4.1, statically enforced rather than textually documented — a small but cumulative gain in the discipline of analytic-number-theoretic formalization.

A Lean file inventory

File	Layer	Status
TS6/Exact/FiniteCharacterVariance.lean	TS6-A	GREEN, 0 sorry
TS6/Exact/DirichletVariance.lean	TS6-B	GREEN, 0 sorry
TS6/Exact/CenteringShift.lean	TS6-C	GREEN, 0 sorry
TS6/Structure/RankinSelbergDecomp.lean	TS6-D	GREEN, 0 sorry
TS6/Effective/AveragedBounds.lean	TS6-E/F/G	GREEN CONDITIONAL, 0 sorry

Table 4: TS6 v4.1 file inventory. Source available at <https://github.com/seisner1967-hash/ts6-lean-formalization>.

B Key Lean signatures (verbatim from the source)

B.1 TS2 exact identity

```
theorem TS2_exact_identity
  (a : N -> C) (q : N) (hq : 2 <= q) (S : Finset N) :
  AWstar a q S = CW a q S
```

B.2 Rankin–Selberg decomposition

```
theorem rankin_selberg_decomposition
  (a : N -> C) (q : N) [NeZero q] (hq : 2 <= q) (S : Finset N) :
  AWstar a q S =
    DW a q S - (Complex.normSq (T_0 a q S)) / (Nat.totient q : R) + (SW a q S).re
```

B.3 Parseval/Plancherel for finite abelian groups

```
theorem parseval_identity
  {G : Type*} [CommGroup G] [Fintype G] [DecidableEq G]
  (f : G -> C) :
  variance f = characterSideMS f
```

B.4 Typed Large Sieve interface

```
structure LargeSieveInequality where
  bound :
    forall (a : N -> C) (x : R) (Q : N) (S : Finset N),
      2 <= x ->
      (forall n in S, ((n : R) <= 2*x)) ->
      weightedPrimeCappedSum a Q S <=
        ((Q : R)^2 + 2*x) * coeffEnergy a S
```

B.5 TS4 weighted effective bound (GREEN-conditional)

```
theorem TS4_weighted_effective_bound
  (chebyshev : ChebyshevThetaBound)
  (largeSieve : LargeSieveInequality)
  (Wi : WeightInput) (x : R) (Q : N) (S : Finset N)
  (hx : 2 <= x) (hQ : 2 <= Q)
  (hS : forall p in S, p.Prime /\ x/2 <= p /\ (p : R) <= 2*x) :
  weightedPrimeCappedSum (weightCoeffs Wi.W x) Q S <=
    4 * WsupNormSq Wi.W * ((Q : R)^2 + 2*x) * x * Real.log (2*x)
```

C Freeze manifest excerpt

```
TS6 v4.1 - Lean 4 GREEN CONDITIONAL Freeze Manifest
Date: 2026-05-03

Lean toolchain: leanprover/lean4:v4.16.0
Mathlib revision: a6276f4c6097675b1cf5ebd49b1146b735f38c02
Build status: SUCCESS

Status table:
  TS6-A FiniteCharacterVariance.lean  GREEN          0 sorry
  TS6-B DirichletVariance.lean        GREEN          0 sorry
  TS6-C CenteringShift.lean           GREEN          0 sorry
  TS6-D RankinSelbergDecomp.lean      GREEN          0 sorry
  TS6-E/F/G AveragedBounds.lean      GREEN CONDITIONAL 0 sorry

Primary REAL sorry: 0
Primary REAL axiom: 0
sorryAx in cascade: NONE

#print axioms (effective theorems):
[propext, Classical.choice, Quot.sound]
```

D Reproduction commands

The following self-contained command sequence reproduces the verification of TS6 v4.1 from a clean shell:


```
# Clone the public archival repository
git clone https://github.com/seisner1967-hash/ts6-lean-formalization
cd ts6-lean-formalization

# Verify the tag
git checkout v4.1

# Build (requires elan: https://leanprover-community.github.io/get_started.html)
lake build TS6
# Expected: Build SUCCESS

# Verify zero primary sorry (POSIX-portable form)
grep -rnE "[[:space:]]*sorry[[:space:]]*" TS6/
# Expected: no output
```

To reproduce the audit of kernel axioms used by the three GREEN-conditional effective theorems, create a minimal Lean file at the repository root and run it through the project environment:

```
cat > Axioms.lean <<'EOF'
import TS6.Effective.AveragedBounds

#print axioms TS6.Effective.TS4_weighted_effective_bound
#print axioms TS6.Effective.TS4_unweighted_effective_bound
#print axioms TS6.Effective.TS3_first_effective_bound
EOF

lake env lean Axioms.lean
```

Expected output, verbatim, for each of the three theorems:

```
'TS6.Effective.TS4_weighted_effective_bound' depends on axioms:
[proptext, Classical.choice, Quot.sound]
'TS6.Effective.TS4_unweighted_effective_bound' depends on axioms:
[proptext, Classical.choice, Quot.sound]
'TS6.Effective.TS3_first_effective_bound' depends on axioms:
[proptext, Classical.choice, Quot.sound]
```

The freeze manifest `TS6_FROZEN_v4.1_2026-05-03.txt` (root of the repository) records SHA-256 hashes of the five primary source files for byte-level verification. The audit report `reports/report_20260503T124108Z.md` (also at the root) contains the full output of `horizon-proof-agent v0.1.4` for this state.

Reference state. Tag `v4.1`; commit `02a6ffa`; Lean toolchain `leanprover/lean4:v4.16.0`; Mathlib revision `a6276f4c6097675b1cf5ebd49b1146b735f38c02`.

End of TS6 v4.1.